

Definitional Equalities for Substitutions using Rewrite Rules

Anonymous Author(s)

Abstract

Choosing a representation of variable binding is a central problem in mechanizing programming language metatheory. De Bruijn indices provide a canonical representation, but developments built on them require numerous lemmas about how substitutions compose and how they pass under binders, and every one of those lemmas has to be applied by hand or by a tactic.

The σ -calculus makes parallel substitutions first-class and normalizes them by rewriting. Modeling substitutions as functions turns its rules into theorems, which is what libraries such as AUTOSUBST exploit, but the resulting equalities stay propositional.

We register the σ -calculus as rewrite rules in Agda, so that substitutions are normalized during conversion rather than by a tactic. Two requirements pull against each other. The rules are provable only if the operations compute, and eligible for rewriting only if the involved definitions are stuck.

We identify which σ -calculus version is eligible for rewriting, and derive a rewrite system where renamings stay first-class, because proof assistants need them to define substitution by structural recursion. We supply a code generator that emits the whole development from a syntax description. Local confluence is checked in Agda and termination in AProVE. To test the rules at scale we solve the POPLmark and POPLmark Reloaded challenges. Across all their parts, only one substitution law is left to apply by hand.

Keywords: Explicit Substitutions, Term Rewrite Systems, Agda

1 Introduction

Handling variable binding remains a central challenge in mechanizing programming language metatheory. Compared to pen-and-paper proofs, binders in proof assistants are a notorious source of uninteresting technicalities and boilerplate. Many representations have been proposed to contain them, among them locally nameless binders [12], nominal syntax [39, 40], higher-order abstract syntax [29], and de Bruijn indices [17]. De Bruijn syntax makes α -equivalent terms syntactically identical, and the syntax itself is easy to define. A development built on de Bruijn indices still pays for a long list of lemmas about substitution composition and about the interaction of substitution with binders.

On paper, substitution usually replaces a single variable. In a proof assistant one instead works with parallel substitutions, which map variables to terms and replace all of them simultaneously. The σ -calculus [1] makes such maps first-class objects with primitive syntactic operations for identity, weakening, extension and composition, together with a rewrite system that normalizes expressions built from them. The de Bruijn algebra [33] is the canonical semantic model for those operations, a two-sorted algebra over de Bruijn terms and substitutions, with substitutions modeled as total functions from variables to terms. In that model the rules are theorems, which is what makes them usable for equational reasoning in proof assistants. AUTOSUBST [34] and its successor AUTOSUBST 2 [38] generate the primitive operations, the proofs of the laws, and a tactic that normalizes substitutions inside a goal. AUTOSUBST 2 additionally supports intrinsically scoped syntax, packages substitutions over several sorts into vectors, and promotes renamings, the substitutions that map variables to variables, to first-class objects of the equational theory.

The equalities obtained by AUTOSUBST are propositional. Normalization becomes a step the user takes, so every goal that mentions a substitution has to be brought into normal form before it can be closed. In Rocq that step is close to automatic, because AUTOSUBST supplies a tactic that performs the whole normalization procedure, but what the tactic produces is still a propositional equality. Recent support for user-defined rewrite rules in Agda [14, 15] and Rocq [25] removes the step. A proof of a propositional equality can be registered as a rule, and its left-hand side then reduces to its right-hand side, so the equality holds definitionally. The difference can be large for intrinsically typed syntax, where a substitution appears in a type index and every use of a propositional law becomes a transport [10]. Consistency of rewrite rules is retained as long as the registered rules are supported by proofs and the resulting system is confluent and terminating [15]. Agda applies a rewrite rule by matching a stuck term (i.e. a term that does not reduce) against the left-hand side of the registered proof [2].

Whether the left-hand side of a substitution law from the σ -calculus is eligible for rewriting is decided by how the operations in it are defined. The same choice decides whether that law is provable. Modeled as functions from variables to terms or as vectors of terms, the σ -calculus operations compute and the rule set of the rewrite system is provable, but a definition that computes also introduces unwanted reductions inside left-hand sides of the propositional equalities. If we decide to model the operations as constructors,

CPP '27,

2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/XXXXXXX.XXXXXXX>

every left-hand side is stuck by construction, but we cannot derive proofs for the laws, because a constructor has nothing to compute. Wadler takes a minimal version of the second route, making only weakening a constructor in the syntax, thereby successfully avoiding renamings [41]. But weakening then does not push inwards into terms, and the definitional equalities that would push it there are lost and unrecoverable.

We choose a definition using functions whose laws are provable and block their reduction artificially, when needed. Agda's opaque definitions [19] give the required control over the reduction behavior of definitions. A function declared opaque does not reduce outside its block, and an unfolding instruction lets a later block see through it again. The operations are ordinary functions where the proofs need them and stuck definitions where the rules match them.

Opacity settles how the operations are defined, and leaves open which σ -calculus variant can be registered as rewrite rules. Not every variant can be made definitional in Agda. We also want renamings to be first class, because a proof assistant needs them to define substitution by structural recursion.

Contributions

- **A σ -calculus for proof assistants.** We give an σ -calculus variant with first-class renamings, that we make definitional in a multi-sorted, intrinsically scoped setting using Agda. Every rule is proved before it is registered, so logical consistency is preserved.
- **A metatheoretic study of the rewrite system.** Agda's built-in checker can validate local confluence, and using an external checker we can prove termination and confluence of the first-order export.
- **A code generator.** We supply a script that emits the entire infrastructure from a higher-order abstract syntax description.
- **An evaluation of the approach.** We solve the POPLmark Challenge, Parts 1A, 1B, 2A, 2B and 3, and POPLmark Reloaded, on definitions and the rewrite system generated by the script. Only one substitution fact remains propositional.

Structure

In section 2 we explain the scoped syntax setting we work in and define renaming and substitution over it. In section 3 we state the conditions Agda puts on a rewrite rule, identify the σ -calculus variant that meets them, derive the rewrite system that follows, and exercise it on the metatheory of System F. In section 4 we establish confluence and termination. In section 5 we explain how to generate the infrastructure from a syntax description and exercise the rewrite system at scale on the POPLmark challenge. Section 6 discusses related work, section 7 collects future work, and section 8 concludes.

2 Syntax and Substitution

The development is carried out in Agda throughout. Proofs in Agda are significantly easier when terms are definitionally equal, that is, when they reduce to the same term. When they do not, users must resort to manual equational reasoning with propositional equality, which is cumbersome as Agda lacks good support for automated proof-discharge tactics. Maximizing definitional equalities thus directly reduces manual effort.

This section only establishes the required definitions. We give the multi-sorted scoped syntax of the paper (section 2.1) and define renaming and substitution over it (section 2.2). Two features of these definitions are important for what section 3 can register as rewrite rules. Variables form a sort of their own, generated by zero and successor, as is standard and required to represent all variables. Renamings are defined separately, because substitution is not structurally recursive without them.

2.1 Multi-Sorted and Scoped Syntax

Many programming languages feature several syntactic categories. For instance, System F [18, 30] has three: expressions, types, and kinds. Each usually gets its own datatype definition. Figure 1 (left) shows the standard intrinsically scoped implementation, where types are indexed by the number of type variables and expressions by the numbers of type and expression variables. Variables are correct-by-construction de Bruijn indices of type `Fin n`, the natural numbers smaller than n . Thus, scoped syntax rules out scoping errors without committing to full intrinsic typing.

The standard representation forces one substitution operation per pair of sorts, such as type-in-type, type-in-expression and expression-in-expression. This is the representation AUTOSUBST 2 chooses, and it explains why their rewrite system needs to be extended with vector substitutions, which bundle the substitutions over the different sorts and come with normalization rules of their own.

To avoid this combinatorial explosion we unify the syntax along two orthogonal dimensions, shown in fig. 1 (right). Variables and terms form a single datatype $S \vdash m \ s$, indexed by a `Mode` m , a target `Sort` s and a `Scope` S , a list of sorts. Variables of type $S \ni s$ are represented across all sorts at once, the length of S being the number of variables in scope and the position of s in S the de Bruijn index. Renamings and substitutions defined over this syntax then operate uniformly across sorts, and vector substitutions become unnecessary.

Recent work extends it to the maps over that syntax [6, 27], deriving renaming and substitution from a single traversal that abstracts over what a variable is mapped to, a variable or a term. Our syntax admits the same treatment, and some laws of section 3 would then follow uniformly for renamings and substitutions rather than twice in similar shape. We keep

```

221 data Kind : Set where
222   ★ : Kind
223
224 data Type (n : Nat) : Set where
225   ' _ : Fin n → Type n
226   ∀[α:_]_ : Kind → Type (suc n) → Type n
227   _⇒_ : Type n → Type n → Type n
228
229 data Expr (n m : Nat) : Set where
230   ' _ : Fin m → Expr n m
231   λx_ : Expr n (suc m) → Expr n m
232   Λα_ : Expr (suc n) m → Expr n m
233   ' _ : Expr n m → Expr n m → Expr n m
234   ' _ : Expr n m → Type n → Expr n m
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275

```

```

276 data Sort : Set where
277   expr type kind : Sort
278   Scope = List Sort
279
280 data Mode : Set where V T : Mode
281
282 data _⊢[_]_ : Scope → Mode → Sort → Set
283
284 _⊢_ = _⊢[ T ]_
285 _⊢_ = _⊢[ V ]_
286
287 data _⊢[_]_ where
288   zero : (s :: S) ⊢ s
289   suc : S ⊢ s → (s' :: S) ⊢ s
290   ' _ : S ⊢ s → S ⊢ s
291   λx_ : (expr :: S) ⊢ expr → S ⊢ expr
292   Λα_ : (type :: S) ⊢ expr → S ⊢ expr
293   ∀[α:_]_ : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
294   ' _ : S ⊢ expr → S ⊢ expr → S ⊢ expr
295   ' _ : S ⊢ expr → S ⊢ type → S ⊢ expr
296   ' _ : S ⊢ type → S ⊢ type → S ⊢ type
297   * : S ⊢ kind
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330

```

Figure 1. System F as classically scoped syntax (left) and as the multi-sorted scoped syntax we use (right)

renaming and substitution apart for clarity of presentation, and because several of our rules treat the two differently.

2.2 Renaming and Substitution

Instantiation, the operation that applies a substitution to a term, when defined by mutual recursion with composition is not structurally recursive. As is standard, we follow Adams [3] and define instantiation with renamings first. Renamings then run through the whole development, and proofs about them need the same reasoning we want to automate for substitutions. We will later follow AUTOSUBST 2 and put both into the equational theory and relate the two, when possible.

We model renamings and substitutions as functions (fig. 2). A renaming ($_ \rightarrow^R _$) takes every variable to a variable of the same sort, changing only the scope. A substitution ($_ \rightarrow^S _$) takes it to a term of that sort instead. We call either of them a *map* where the distinction does not matter. The operations over maps are functions as well, and all of them are placed in an *opaque* block. They stay stuck, and we unfold them only in the proofs that need their computational behavior.

Modeling maps as functions is a choice. A vector of images, one per variable in the domain scope, would do as well, and would need the same opacity. Extension is a constructor there and stuck by itself, but every other operation is still a definition that unfolds. We take functions because we find them easier to think about, at the price of postulating function extensionality. We talk more about the vector model in ??

The identity (id^R) takes every variable to itself, and takes its sort argument explicitly because Agda frequently fails to infer it. Weakening (wk^R) takes a variable to its successor. Extension ($_ \rightarrow^R _$) adds an image for a newly bound variable, and lifting ($_ \uparrow^R _$) fixes that variable and weakens the images of the rest, which is what lets a renaming pass under a binder.

Instantiation ($_ \llbracket _ \rrbracket^R$) traverses a term and lifts whenever it enters a binder. It is defined on variables as well, so that a lookup is written $x \llbracket \xi \rrbracket^R$ rather than as the application $\xi _ x$. We only show three of its clauses, for a variable, a lambda binding and an application, and omits the five that are evident congruences. Composition ($_ \circ^R _$) looks a variable up in the first renaming and renames the result with the second, hence it's only ordinary function composition in disguise.

The coercion operator ($_ \langle _ \rangle$) embeds a renaming in a substitution, wrapping each image variable in the variable constructor. The remaining substitution operations are the same one level up, with terms in place of variables, and the figure lists them in that order. Extension ($_ \rightarrow^S _$) reads as before. Lifting ($_ \uparrow^S _$) weakens the images of the rest with the weakening renaming, which is what keeps the recursion structural. Instantiation ($_ \llbracket _ \rrbracket^S$) and composition ($_ \circ^S _$) read as before as well, except that a lookup now returns a term. The identity (id^S) and weakening (wk^S) substitutions are non-opaquely defined in terms of the coercion operator.

$\rightarrow^R : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set}$
 $S_1 \rightarrow^R S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \ni s$

opaque

$\text{id}^R : S \rightarrow^R S$

$\text{id}^R _ x = x$

$\text{wk}^R : \forall s' \rightarrow S \rightarrow^R (s' :: S)$

$\text{wk}^R _ x = \text{succ } x$

$_ \cdot^R : S_2 \ni s \rightarrow S_1 \rightarrow^R S_2 \rightarrow (s :: S_1) \rightarrow^R S_2$

$(x \cdot^R \xi) _ \text{zero} = x$

$(_ \cdot^R \xi) _ (\text{succ } x) = \xi _ x$

$_ \uparrow^R : S_1 \rightarrow^R S_2 \rightarrow \forall s \rightarrow (s :: S_1) \rightarrow^R (s :: S_2)$

$(\xi \uparrow^R _) _ \text{zero} = \text{zero}$

$(\xi \uparrow^R _) _ (\text{succ } x) = \text{succ } (\xi _ x)$

$_ \llbracket^R : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^R S_2 \rightarrow S_2 \vdash [m] s$

$_ \llbracket^R \{m = V\} x \xi = \xi _ x$

$(_ x) \llbracket \xi \rrbracket^R = _ (x \llbracket \xi \rrbracket^R)$

$(\lambda x e) \llbracket \xi \rrbracket^R = \lambda x (e \llbracket (\xi \uparrow^R _) \rrbracket^R)$

$(e_1 \cdot e_2) \llbracket \xi \rrbracket^R = (e_1 \llbracket \xi \rrbracket^R) \cdot (e_2 \llbracket \xi \rrbracket^R)$

-- ...

$_ \circ^R : S_1 \rightarrow^R S_2 \rightarrow S_2 \rightarrow^R S_3 \rightarrow S_1 \rightarrow^R S_3$

$(\xi_1 \circ^R \xi_2) _ x = (\xi_1 _ x) \llbracket \xi_2 \rrbracket^R$

$\rightarrow^S : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set}$
 $S_1 \rightarrow^S S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \vdash s$

opaque

$\langle _ \rangle : S_1 \rightarrow^R S_2 \rightarrow S_1 \rightarrow^S S_2$

$\langle \xi \rangle _ x = _ (x \llbracket \xi \rrbracket^R)$

$_ \cdot^S : S_2 \vdash s \rightarrow S_1 \rightarrow^S S_2 \rightarrow (s :: S_1) \rightarrow^S S_2$

$(t \cdot^S \sigma) _ \text{zero} = t$

$(t \cdot^S \sigma) _ (\text{succ } x) = \sigma _ x$

$_ \uparrow^S : S_1 \rightarrow^S S_2 \rightarrow \forall s \rightarrow (s :: S_1) \rightarrow^S (s :: S_2)$

$(\sigma \uparrow^S _) _ \text{zero} = _ \text{zero}$

$(\sigma \uparrow^S _) _ (\text{succ } x) = (\sigma _ x) \llbracket \text{wk}^R _ \rrbracket^R$

$_ \llbracket^S : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^S S_2 \rightarrow S_2 \vdash s$

$_ \llbracket^S \{m = V\} x \sigma = \sigma _ x$

$(_ x) \llbracket \sigma \rrbracket^S = \sigma _ x$

$(\lambda x e) \llbracket \sigma \rrbracket^S = \lambda x (e \llbracket (\sigma \uparrow^S _) \rrbracket^S)$

$(e_1 \cdot e_2) \llbracket \sigma \rrbracket^S = (e_1 \llbracket \sigma \rrbracket^S) \cdot (e_2 \llbracket \sigma \rrbracket^S)$

-- ...

$_ \circ^S : S_1 \rightarrow^S S_2 \rightarrow S_2 \rightarrow^S S_3 \rightarrow S_1 \rightarrow^S S_3$

$(\sigma_1 \circ^S \sigma_2) _ x = (\sigma_1 _ x) \llbracket \sigma_2 \rrbracket^S$

$\text{id}^S : S \rightarrow^S S$

$\text{id}^S = \langle \text{id}^R \rangle$

$\text{wk}^S : \forall s' \rightarrow S \rightarrow^S (s' :: S)$

$\text{wk}^S s' = \langle \text{wk}^R s' \rangle$

Figure 2. The operations for renaming (left) and for substitution (right)

3 Rewrite Rules for Renamings and Substitutions

Agda accepts an equation as a rewrite rule only under conditions, and not every σ -calculus variant can be presented as a rule set whose equations all meet them. We state the conditions (section 3.1), identify a variant that Agda accepts (section 3.2), and derive the rewrite system that follows with first class renamings (section 3.3). We put the resulting system at work on the metatheory of System F (section 3.5). Throughout, the multi-sorted scoped System F is the running example, but the construction applies to any multi-sorted scoped syntax.

3.1 Rewrite Rules in Agda

User-defined rewrite rules [14, 15] extend Agda's normalization-by-evaluation approach for definitional equalities by treating propositional equalities as reduction rules. A proof of $f p_1 \dots p_n \equiv v$ is eligible for rewriting under three conditions.

- f is a postulate, a defined function or a constructor.

- The left-hand side is stuck.
- Every variable bound by the rule occurs on the left-hand side, and occurs there in pattern position.

The last condition serves matching. Applying a rule means matching its left-hand side against a stuck term, and that match has to determine every variable the rule binds. Matching is higher-order [2], that is, a variable bound by the rule may stand for a function and occur applied on the left-hand side. In general that problem is undecidable, and the requirement that all bound variables need to appear in *pattern position* is the restriction that makes it decidable. A bound variable appears in pattern position if it is either not applied at all, or applied to distinct bound variables and to nothing else. So $\xi s x$ qualifies, where ξ , s and x are all bound by the rule, whereas $\xi s (\text{succ } x)$ does not.

For example, renaming a term by the identity leaves it unchanged. The law is proved by induction on the term, so it does not hold definitionally on its own:


```

441 [id] : ∀ {n} (t : Tm n) → t [ idR ] ≡ t
442 [id] (‘ x)      = refl
443 [id] (t1 · t2) = cong2 _·_ ([id] t1) ([id] t2)
444 -- ...

```

It meets the three conditions: the left-hand side is stuck on the defined function `_[]`, because no clause matches a variable term, and its only bound variable t occurs there unapplied. Once it is registered via `{-# REWRITE [id] #-}`, Agda treats $t [id^R]$ as definitionally equal to t :

```

451 _ : ∀ {n} {t : Tm n} → t [ idR ] ≡ t
452 _ = refl

```

Beyond the syntactic restriction, all rewritten equations as a whole have to be well behaved, in two respects. Subject reduction needs confluence, and consistency needs the rules to be proven propositionally [15, Sect. 3 and 6.4]:

“The rewriting system defined by the rewrite rules together with the standard reduction rules of type theory should be confluent.” ... “if we can provide an instantiation of the rewrite block that satisfies each of the rewrite rules propositionally, then we can ensure consistency.”

Neither obligation asks for strong normalization. We discharge both separately. Every rule is proved before it is registered (section 3.3). For confluence we rely on Agda’s local-confluence check, which computes the critical pairs of the rewrite system and reports those it cannot join. Local confluence implies confluence only for a terminating system, so we also need to establish termination (section 4) and conclude confluence by Newman’s lemma [9, 28]. That last step has to leave Agda, because Agda cannot check termination. A non-terminating rule set can make the type checker loop, but only on a term that triggers the divergence, so it may survive undetected.

3.2 Which Variant Can Be Registered

Explicit-substitution calculi (TODO: called σ -calculi throughout) turn substitutions into syntax, so that $t [\sigma]$ is a term and σ itself can be composed, extended and pushed under binders by rewriting. Three of them matter here.

Not every version of the σ -calculus is a candidate for rewriting. The original [1] is confluent on ground terms only, and open terms, those with metavariables, are the setting of any development with global rewrite rules. The two variants that repair this differ on the η -laws. Those identify every map with its first entry followed by the rest, which is surjective pairing for substitutions. Whether a variant can keep them depends on whether its variables are a sort of their own, generated inductively by zero and successor. σ_{SP} [16] has no such sort, and its n -th variable is the first one weakened n times, so it can keep η and be complete for the de Bruijn algebra [33, 37]. AUTOSUBST keeps η alongside

native inductive variables, because it never installs its equations as conversion, and a tactic may apply a propositional lemma to any term, stuck or not.

$\sigma_{\uparrow}$ [16, 20] generates its variables inductively, so a weakened variable reduces to its successor, and that rule destroys the left-hand side of η . Our syntax has that sort, by the choice of section 2.1, and we need it. Without it the n -th variable can only be written by applying weakening, so weakening becomes a constructor, which is Wadler’s choice. We therefore take $\sigma_{\uparrow}$. Registering it is what makes substitution definitional for us. It costs completeness, which first-class renamings put out of reach anyway, because they take the theory outside the one σ_{SP} was proved complete for.

That left-hand side, $(\text{zero}[\sigma]) \cdot (\text{wk} \circ \sigma)$, mentions σ twice, so any rule that rewrites one occurrence destroys the pattern for the other. In $\sigma_{\uparrow}$ lifting is primitive, and its 23 rules, tabulated by Hardin et al. [20], fall into the applied rules, where a variable meets a map, the traversal rules, one per constructor, the map algebra, and a family of $\circ$ -companions that exists only because associativity is directed.

Primitive lifting and the absence of η stand or fall together. Eliminate lifting and lift-id, $\uparrow \text{id} = \text{id}$, stops holding by computation, because $\uparrow$ then unfolds to $\text{zero} \cdot (\text{id} \circ \text{wk})$ and only η -id reconciles the two.

The price of dropping η is four laws, the two η -laws once for renamings and once for substitutions, which we prove but do not register. Section 5.2 reports what that costs in practice.

One further ingredient appears in none of these calculi. Defining instantiation for a substitution requires pushing under binders, which requires weakening the terms in the substitution, which is instantiation again, a loop no termination checker accepts. The standard repair is to define renamings first [3]. AUTOSUBST 2 takes the further step of making renamings first-class and extending the equational theory with rules that relate the two worlds [38]. The extended system is no longer complete, and its confluence was not formally verified [37].

For the semantics we use the de Bruijn algebra, which models substitutions as functions and in which the rules are theorems rather than postulates [34]. Variables are natural numbers, terms are $t := x \mid \lambda. t \mid t_1 t_2$, and a substitution maps variables to terms. Reading σ as a list $[t_0, t_1, \dots]$ motivates the operations:

$$\begin{aligned}
 (t \cdot \sigma)(\text{zero}) &:= t & \text{id}(x) &:= x \\
 (t \cdot \sigma)(\text{suc } x) &:= \sigma(x) & \text{wk}(x) &:= \text{suc } x
 \end{aligned}$$

Instantiation is then defined mutually with composition and lifting:

$$\begin{aligned}
 (\lambda. t) [\sigma] &= \lambda. (t [\uparrow \sigma]) & (\sigma_1 \circ \sigma_2)(x) &= \sigma_1(x) [\sigma_2] \\
 (t_1 t_2) [\sigma] &= (t_1 [\sigma]) (t_2 [\sigma]) & \uparrow \sigma &:= \text{zero} \cdot (\sigma \circ \text{wk})
 \end{aligned}$$

Lifting fixes the bound variable and weakens the rest, and composition looks a variable up in the first map and instantiates the result with the second.

3.3 The Sigma Calculus Rules

Seventy-two rules is too many to read one at a time. Reading them that way would also hide what matters about the number, which is that it is determined. The system is a small base closed under three completion operators, and every rule belongs to exactly one group. Figure 3 shows the substitution-world half as Agda equalities, and table 1 gives the groups.

Table 1. The 72 registered rules by group. The four groups above the line are the calculus itself. The three below it are what our design decisions cost.

#	group	what it is for
16	traversal	push an application through one constructor, the only signature-dependent group, two rules per constructor
9	lookup	evaluate a constructor-headed map at a variable, that is, the definitions of wk , $\cdot$ and $\uparrow$
16	map algebra	normalize a composed map, once per world
7	fusion	turn two traversals into one, in all four renaming/substitution combinations, the two mixed cases being AUTOSUBST 2's compRenSubst and compSubstRen [38]
7	continuation	assoc right-nests $\circ$, so a rule matching a non-variable right operand can no longer see it inside a chain
8	mode V	a lookup cannot evaluate an accumulator, so at mode V the map must be driven back to constructor-headed form
9	coercion	$\langle \cdot \rangle$ -lift and $\langle \cdot \rangle$ -comp erase the $\uparrow$ and $\circ$ heads that the σ -rules match on, so every such rule needs a coerced image

The four groups above the line in table 1 are the calculus. The three below are the price of design decisions, and each comes with a side condition saying when an image is *not* needed. Those side conditions are what make the count predictive rather than empirical, so we work one of them out.

Orienting associativity right-nests $\circ$, so a rule whose right operand is not a metavariable stops matching once a continuation is appended. Take lift-wk and overlap it with assoc on $(\langle \text{wk} \rangle \circ (\uparrow \sigma)) \circ \tau$. Reducing the inner redex first gives $(\sigma \circ \langle \text{wk} \rangle) \circ \tau$, which assoc takes to $\sigma \circ (\langle \text{wk} \rangle \circ \tau)$. Reducing with assoc first gives $\langle \text{wk} \rangle \circ ((\uparrow \sigma) \circ \tau)$, where nothing matches any more. The pair is closed by registering exactly that term as the continuation image of lift-wk , and six other base rules need the same treatment.

interaction, $\langle \text{wk} \rangle \circ (t \cdot \sigma) = \sigma$, has the identical overlap and needs no image. Its continued redex is $\langle \text{wk} \rangle \circ ((t \cdot \sigma) \circ \tau)$, and here the right operand does reduce. dist turns it back into extension form, after which interaction matches again.

That is the side condition. An image is needed when the right operand is not a metavariable *and* the continued redex has no inner reduction. Five rules satisfy the first half and fail the second, namely comp-id , in both worlds, interaction, lift-cons and its coercion image, and the supplement checks each escape as a refl assertion. The rule set matches the predicate exactly, absences included.

Three rules carry the design.

coincidence, $t[\langle \xi \rangle]^S = t[\xi]^R$, is where the direction is chosen. It makes the renaming world the normal form, so renaming-only work never leaves it. Everything in the coercion group follows from that direction.

compositionality^{RR} exists twice, pointing in opposite directions:

$$\begin{aligned} (t[\xi_1])[\xi_2] &= t[\xi_1 \circ \xi_2] && \text{at mode T, fold} \\ x[\xi_1 \circ \xi_2] &= (x[\xi_1])[\xi_2] && \text{at mode V, push} \end{aligned}$$

This is the only place where the two modes disagree on a direction, and they must. Renaming preserves the mode, so $x[\xi]$ is again a variable and again a subject for the applied rules. Folding at a variable would leave a pending composition that no lookup rule can evaluate, and it would overlap $x[\text{wk}] = \text{suc } x$ unjoinably. At a term, accumulating is instead what keeps the traversal linear. In the substitution world the question does not arise, because a substituted variable is a term and no applied rule can match it.

$\langle \cdot \rangle$ -comp is the one completion image in the system that cannot be taken. The closure demands it, but the general image is the exact inverse of $\langle \cdot \rangle$ -split, so registering both loops. What we register instead is that image restricted to the prefixes on which the renaming world can still make progress. There are three of them, interaction^R , lift-wk^R and lift-dist-comp^R , and they are exactly the three renaming-world rules that need continuation images of their own. The same set answers both questions.

Nine further laws are proved but not registered. Four are the η -laws, for the left-linearity reason of section 3.2. Two are dist^R and lift-cons^R , whose variable instances collide with the lookup rules under *push at V*. Two are the definitions of lifting, which we keep primitive. The last, lift-id , is subsumed. Its left-hand side is a strict instance of $\langle \cdot \rangle$ -lift's, which already sends it to the same normal form.

All rules are proved before being registered, which is what preserves consistency. The proofs unfold the opaque definitions and use function extensionality, which is conservative over Agda's theory [21]. We refer the reader to the supplement.

3.4 Why the Definitions Must Not Unfold

Blocking the reduction of the operations is what lets us register every rule. We now explain why, and why the alternatives fail. The block is also permanent. An **unfolding** declaration

$\text{coincidence-var} : x [\langle \xi \rangle]^S \equiv (x [\xi]^R)$
 $\text{def--}^S\text{-zero} : \text{zero} [(t \cdot^S \sigma)]^S \equiv t$
 $\text{def--}^S\text{-suc} : (\text{suc} \{s' = s'\} x) [(t \cdot^S \sigma)]^S \equiv x [\sigma]^S$
 $\text{def-}\uparrow^S\text{-zero} : \text{zero} [(\sigma \uparrow^S s)]^S \equiv \text{zero}$
 $\text{def-}\uparrow^S\text{-suc} : (\text{suc } x) [(\sigma \uparrow^S s)]^S \equiv x [(\sigma \circ^S \langle \text{wk}^R s \rangle)]^S$
 $\text{interact} : \langle \text{wk}^R s \rangle \circ^S (t \cdot^S \sigma) \equiv \sigma$
 $\text{comp-id}_l : \langle \text{id}^R \{S_1\} \rangle \circ^S \sigma \equiv \sigma$
 $\text{comp-id}_r : \sigma \circ^S \langle \text{id}^R \rangle \equiv \sigma$
 $\text{lift-wk} : \langle \text{wk}^R s \rangle \circ^S (\sigma \uparrow^S s) \equiv \sigma \circ^S \langle \text{wk}^R s \rangle$
 $\text{assoc} : (\sigma_1 \circ^S \sigma_2) \circ^S \sigma_3 \equiv \sigma_1 \circ^S (\sigma_2 \circ^S \sigma_3)$
 $\text{dist} : (t \cdot^S \sigma_1) \circ^S \sigma_2 \equiv (t [\sigma_2]^S) \cdot^S (\sigma_1 \circ^S \sigma_2)$
 $\text{lift-cons} : (\sigma \uparrow^S s) \circ^S (t \cdot^S \tau) \equiv t \cdot^S (\sigma \circ^S \tau)$
 $\text{lift-dist-comp}^{SS} : ((\sigma_1 \uparrow^S s) \circ^S (\sigma_2 \uparrow^S s)) \equiv ((\sigma_1 \circ^S \sigma_2) \uparrow^S s)$

$\text{coincidence} : \forall (t : S \vdash s) (\xi : S \rightarrow^R S_2) \rightarrow t [\langle \xi \rangle]^S \equiv t [\xi]^R$
 $\langle \rangle\text{-comp} : \langle \xi_1 \rangle \circ^S \langle \xi_2 \rangle \equiv \langle \xi_1 \circ^R \xi_2 \rangle$
 $\langle \rangle\text{-split-}\circ : \langle \xi_1 \circ^R \xi_2 \rangle \circ^S \sigma \equiv \langle \xi_1 \rangle \circ^S (\langle \xi_2 \rangle \circ^S \sigma)$
 $\langle \rangle\text{-lift} : (\langle \xi \rangle \uparrow^S s) \equiv \langle \xi \uparrow^R s \rangle$
 $\text{inst-x} : (x) [\sigma]^S \equiv x [\sigma]^S$
 $\text{inst-}\lambda : (\lambda x e) [\sigma]^S \equiv \lambda x (e [(\sigma \uparrow^S _)]^S)$
 $\text{inst-}\Lambda : (\Lambda \alpha e) [\sigma]^S \equiv \Lambda \alpha (e [(\sigma \uparrow^S _)]^S)$
 $\text{inst-}\forall : (\forall [\alpha : k] t) [\sigma]^S \equiv \forall [\alpha : k [\sigma]^S] (t [(\sigma \uparrow^S _)]^S)$
 $\text{inst-}\cdot : (e_1 \cdot e_2) [\sigma]^S \equiv (e_1 [\sigma]^S) \cdot (e_2 [\sigma]^S)$
 $\text{inst-}\bullet : (e \bullet t) [\sigma]^S \equiv (e [\sigma]^S) \bullet (t [\sigma]^S)$
 $\text{inst-}\Rightarrow : (t_1 \Rightarrow t_2) [\sigma]^S \equiv (t_1 [\sigma]^S) \Rightarrow (t_2 [\sigma]^S)$
 $\text{inst-}\ast : \ast \{S = S\} [\sigma]^S \equiv \ast$
 $\text{compositionality}^{SS} : \forall (x/t : S_1 \vdash [m] s)$
 $\{ \sigma_1 : S_1 \rightarrow^S S_2 \} \{ \sigma_2 : S_2 \rightarrow^S S_3 \} \rightarrow$
 $(x/t [\sigma_1]^S) [\sigma_2]^S \equiv x/t [(\sigma_1 \circ^S \sigma_2)]^S$

Figure 3. The substitution-world half of the rule set, stated as Agda propositional equalities and then registered as rewrite rules. Left, the lookup rules `def_` and the algebra of composed maps. Right, the coercion family that pushes an embedded renaming back into the renaming world, the per-constructor traversal rules `inst_`, and compositionality. The renaming-world duplicates and the $\circ$ -continuation companions are in the supplement

would put the defining clauses back into the reduction relation, where they compete with the rules, and the confluence check covers the rules alone. We therefore never unfold an operation once its rules are registered.

The immediate problem is that composition unfolds to a lambda abstraction. Unfolded, the left-hand side of `comp-idl` becomes $\lambda s x \rightarrow (\langle \text{id} \rangle s x) [\sigma]$, which is not a stuck. But once composition is blocked, we still have to say how a variable is looked up in a composite.

Our first attempt kept function application for lookup, by rewriting $(\sigma_1 \circ \sigma_2) s x = (x [\sigma_1] [\sigma_2])$. Agda accepts this alongside the interaction rules, but then misbehaves during higher-order pattern unification and stops applying the η -laws, which propositional reasoning still discharges by hand. We think Agda should reject the construction rather than accept it and degrade.

Instead we use the instantiation operator for lookup, which has to be propagated through every rule. Function application and lambda-built maps remain available to the user but yield fewer definitional equalities, so we recommend instantiation for lookup and the two composition operators for everything else.

Using Vectors instead of Functions

A vector-like data type is the other candidate representation, with the map a finite list of terms. Extension alone is a constructor. Identity, weakening, lifting and composition stay defined by recursion over the vector, so their clauses compete with the rules exactly as the function model's do, and opacity is needed all the same.

Nothing is lost on the side of provability, and the supplement carries the experiment. With maps as vectors every law of section 3.3 is a theorem under `--safe`, the four η -laws included. Equality of vectors is equality of data, so that variant assumes nothing, where ours postulates function extensionality. We keep functions as the more intuitive model. The laws are theorems either way, so we can report that the η -laws are never once applied by hand across the POPLmark development.

The designs differ on cost. Every vector operation recurses on the scope, so laws the function model closes by `refl` become inductions the generator has to emit. Making the map a tuple by recursion on the scope instead buys the η -laws definitionally, from Agda's η -rule for records, and forfeits the rules, because the operations then reduce on their own and every rule mentioning one overlaps its defining clauses unjoinably. The trade is made per definition. Transparency buys η and forbids rules. Opacity buys rules and forbids η . We take opacity, and drop η for the independent reason of section 3.2.

Vectors remain the right choice where functions are not available, notably when formalizing type theory in type theory [7], and they are what the uniform renaming/substitution treatments [6, 27] of section 6 use.

The original σ -calculus treats its operations purely syntactically, describing lookup itself through instantiation. The de Bruijn algebra then supplied a semantic model in functions, which made the rules provable and usable for automated propositional reasoning. This work does both at once. The model proves the laws, and the syntactic reading makes them apply.

```

771  $\vdash \lambda : \quad \_ \rightarrow^S \_ : S_1 \rightarrow^S S_2 \rightarrow \text{Ctx } S_1 \rightarrow \text{Ctx } S_2 \rightarrow \text{Set}$ 
772  $(t ::_t \Gamma) \vdash e : (\text{weaken } t') \rightarrow \quad \_ \rightarrow^S \_ \{S_1\} \sigma \Gamma_1 \Gamma_2 = \forall s (x : S_1 \ni s) (t : S_1 \vdash s) \rightarrow$ 
773  $\Gamma \vdash (\lambda x e) : (t \Rightarrow t') \quad \Gamma_1 \ni x : t \rightarrow \Gamma_2 \vdash (x [\sigma]^S) : (t [\sigma]^S)$ 
774  $\vdash \bullet : \quad \text{sub-pres} : \forall \{\sigma : S_1 \rightarrow^S S_2\} \{\Gamma_1 : \text{Ctx } S_1\} \{\Gamma_2 : \text{Ctx } S_2\}$ 
775  $\Gamma \vdash e : (\forall [\alpha : k] t') \rightarrow \quad \{e : S_1 \vdash s\} \{t : S_1 \vdash s\} \rightarrow$ 
776  $\Gamma \vdash t : k \rightarrow \quad \Gamma_1 \vdash e : t \rightarrow \sigma : \Gamma_1 \rightarrow^S \Gamma_2 \rightarrow$ 
777  $(k ::_t \Gamma) \vdash t' : k' \rightarrow \quad \Gamma_2 \vdash (e [\sigma]^S) : (t [\sigma]^S)$ 
778  $\Gamma \vdash (e \bullet t) : (t' [t]_0) \quad \text{sr} : \Gamma \vdash e : t \rightarrow e \hookrightarrow e' \rightarrow \Gamma \vdash e' : t$ 
779  $\beta\text{-}\lambda : \quad \_ \vdash^S : \forall \{\sigma : S_1 \rightarrow^S S_2\} \{\Gamma_1 : \text{Ctx } S_1\} \{\Gamma_2 : \text{Ctx } S_2\} \rightarrow$ 
780  $\text{Val } e_2 \rightarrow \quad \sigma : \Gamma_1 \rightarrow^S \Gamma_2 \rightarrow (t : S_1 \vdash s) \rightarrow$ 
781  $((\lambda x e_1) \cdot e_2) \hookrightarrow (e_1 [e_2]_0) \quad (\sigma \uparrow^S s) : (t ::_t \Gamma_1) \rightarrow^S ((t [\sigma]^S) ::_t \Gamma_2)$ 
782  $\text{wk-cancel} : \forall \{t : S \vdash s\} \{t' : S \vdash s'\} \rightarrow (\text{weaken } t) [t']_0 \equiv t \quad \uparrow[] : \forall \{\Gamma : \text{Ctx } S\} \{e : S \vdash s\} \{t : S \vdash s\} \rightarrow$ 
783  $\text{wk-cancel} = \text{refl} \quad \Gamma \vdash e : t \rightarrow (e \cdot^S \text{id}^S) : (t ::_t \Gamma) \rightarrow^S \Gamma$ 
784  $\text{wk-comm} : \forall \{t : S_1 \vdash s\} \{\sigma : S_1 \rightarrow^S S_2\} \rightarrow \quad \text{wk-comm} = \text{refl}$ 
785  $(\text{weaken } \{s' = s'\} t) [(\sigma \uparrow^S s')]^S \equiv \text{weaken } (t [\sigma]^S)$ 
786  $\text{subst-commute} : \forall \{t : (s' :: S_1) \vdash s\} \{t' : S_1 \vdash s'\} \quad \text{sub-pres } \{\sigma = \sigma\} (\vdash \lambda \vdash e) \vdash \sigma = \vdash \lambda (\text{sub-pres } \{\sigma = \sigma \uparrow^S \_ \} \vdash e (\vdash \uparrow^S \{\sigma = \sigma\} \vdash \sigma \_))$ 
787  $\{ \sigma : S_1 \rightarrow^S S_2 \} \rightarrow \quad \text{-- } \vdash \bullet \text{ concludes at } t' [t]_0, \text{ so the two sides are}$ 
788  $(t [(\sigma \uparrow^S s')]^S) [t' [\sigma]^S]_0 \equiv (t [t']_0) [\sigma]^S \quad \text{-- } (t' [\sigma \uparrow^S \_ ]^S) [t [\sigma]^S]_0 \text{ and } (t' [t]_0) [\sigma]^S.$ 
789  $\text{subst-commute} = \text{refl} \quad \text{-- Discharged by subst-commute.}$ 
790  $\text{sub-pres } \{\sigma = \sigma\} (\vdash \bullet \vdash e \vdash t \vdash t') \vdash \sigma = \vdash \bullet (\text{sub-pres } \{\sigma = \sigma\} \vdash e \vdash \sigma) (\text{sub-pres } \{\sigma = \sigma\} \vdash t \vdash \sigma)$ 
791  $(\text{sub-pres } \{\sigma = \sigma \uparrow^S \_ \} \vdash t' (\vdash \uparrow^S \{\sigma = \sigma\} \vdash \sigma \_))$ 
792  $\text{-- } \vdash \lambda \text{ stores the result type weakened, so the redex is typed at}$ 
793  $\text{-- } (\text{weaken } t_2) [e_2]_0 \text{ where the goal is } t_2. \text{ Discharged by wk-cancel.}$ 
794  $\text{sr } (\vdash \{e_2 = e_2\} (\vdash \lambda \vdash e_1) \vdash e_2) (\beta\text{-}\lambda \ v_2) =$ 
795  $\text{sub-pres } \{\sigma = e_2 \cdot^S \text{id}^S\} \vdash e_1 (\vdash [] \vdash e_2)$ 

```

Figure 4. Everything the three interesting clauses depend on. Top left, the two typing rules and the reduction rule whose types carry a substitution. Top right, the two statements and the notion of a well typed substitution. Middle left, the three laws this creates a demand for, each holding by `refl`; middle right, the two helpers the clauses call. Bottom, the only three clauses of either proof in which a law is spent, with the law named. Every other clause needs none.

3.5 What the Rules Buy

We exercise the rule set on the standard metatheory of the System F of fig. 1, against a typing judgment $\vdash _ : _$ and a call-by-value small-step semantics $\hookrightarrow _$. A context $\text{Ctx } S$ assigns to every variable in scope S a type one sort up, so $\Gamma \vdash e : t$ reads as usual. Figure 4 collects everything the argument below depends on.

Substitution enters this metatheory in exactly three places, shown top left. $\vdash \lambda$ stores the result type of a function *weakened*, because the body is typed in a longer context. $\vdash \bullet$ concludes at a single substitution $t' [t]_0$, because instantiating a polymorphic type substitutes for its bound variable. And

$\beta\text{-}\lambda$ contracts a redex to $e_1 [e_2]_0$. Each of the three puts a substitution somewhere a proof will later have to move it, and each therefore creates a demand for one law.

Top right are the two statements. A substitution is well typed, written $\sigma : \Gamma_1 \rightarrow^S \Gamma_2$, when it sends every variable x typed t in Γ_1 to a term $x [\sigma]^S$ typed $t [\sigma]^S$ in Γ_2 . The type is substituted along with the term, which is what makes the notion compose. `sub-pres` then says that applying such a substitution to a well typed term preserves typing, and `sr` is subject reduction. Both are the expected inductions and use two helpers, shown middle right. $\vdash \uparrow^S$ lifts a well typed substitution under a binder, and is proved from the

corresponding statement for renamings, which is what keeps the recursion structural. $\vdash[]$ turns a well typed term into the well typed single substitution that the two β -cases need.

Middle left are the three laws the three obligations create a demand for. A de Bruijn development states and applies each of them by hand. Here each holds by **refl**, because each is a normalization rather than a lemma: the rules turn both sides into the same normal form, and **refl** sees that they agree. Written out for subst-commute, the classic substitution lemma, the two branches are

$$\begin{aligned}
 & (t \upharpoonright \sigma) [t'[\sigma]]_0 = (t \upharpoonright \sigma) [t'[\sigma] \cdot \text{id}] \\
 & \rightarrow t \upharpoonright (\sigma \circ (t'[\sigma] \cdot \text{id})) && \text{compositionality} \\
 & \rightarrow t \upharpoonright (t'[\sigma] \cdot (\sigma \circ \text{id})) && \text{lift-cons} \\
 & \rightarrow t \upharpoonright (t'[\sigma] \cdot \sigma) && \text{comp-id}_r \\
 & (t \upharpoonright t')_0 [\sigma] = (t \upharpoonright t' \cdot \text{id}) [\sigma] \\
 & \rightarrow t \upharpoonright ((t' \cdot \text{id}) \circ \sigma) && \text{compositionality} \\
 & \rightarrow t \upharpoonright (t'[\sigma] \cdot (\text{id} \circ \sigma)) && \text{dist} \\
 & \rightarrow t \upharpoonright (t'[\sigma] \cdot \sigma) && \text{comp-id}_l
 \end{aligned}$$

Five rules and no user intervention. Every intermediate term above is checked against the endpoint in the supplement, so the trace is the one Agda takes. A normal form is reached when no traversal can be pushed further, no composition can be fused, and no embedded renaming is left in the substitution world. On a closed term the substitution disappears entirely. On an abstract t it survives as a single application to a right-nested, identity-free map, as here.

The bottom of the figure closes the loop, and each clause can now be read against the rule above it. In the $\vdash\lambda$ case the induction hypothesis types the body at $(\text{weaken } t') \upharpoonright \sigma$, because that is where $\vdash\lambda$ put it, while $\vdash\lambda$ demands $\text{weaken } (t' \upharpoonright \sigma)$ to rebuild the abstraction. That is wk-comm. In the $\vdash\bullet$ case the goal is $(t' \upharpoonright t)_0 [\sigma]$ and the three sub-derivations supply $(t' \upharpoonright \sigma) \upharpoonright t [\sigma]_0$, which is subst-commute. In the β -case for $\lambda x_-, \vdash\lambda$ has stored the result type weakened, so **sub-pres** delivers the contractum at $(\text{weaken } t_2) [e_2]_0$ where the goal is plain t_2 , which is wk-cancel.

Every other clause of either proof needs nothing, and for two different reasons. $\vdash\Delta$, $\vdash\cdot$ and $\vdash^*$ conclude at a type the substitution never has to be moved through, so no equation arises. The β -case for $\Delta\alpha_-$ does face one, $t' \upharpoonright t_0$ against $t' \upharpoonright t \cdot \text{id}$, but the two are the same term by the definition of single substitution. Since the three that do arise are conversions as well, every clause of both proofs is a bare constructor application.

The three are not special. Two single substitutions can be exchanged, lifting distributes over composition, and the same again in the renaming world. So do the two mixed laws relating the worlds, which a development without first-class renamings cannot state at all. The supplement checks every one of them, and every one is **refl**.

One case does need help, and the cause is the rewriting itself. In the two β -cases we pass σ explicitly, writing **sub-pres** $\{\sigma = e_2 \cdot^S \text{id}^S\}$. By the time Agda sees the goal, the rules have normalized the type index, so σ can no longer be recovered by unifying against $e \upharpoonright \sigma^S$. Definitional equality and inferability pull against each other here. The more of the index is normalized before unification, the less of it is left to read a metavariable off. The cost is one implicit argument, and no substitution law is applied by hand.

4 Correctness

Section 3.1 left two obligations. Every rule is proved before it is registered, which section 3.3 did. This section establishes confluence, together with the termination that local confluence needs to give it.

Agda's `--local-confluence-check` certifies that every critical pair joins. It says nothing about termination, which Agda does not check for rewrite rules at all. We therefore export the system as a first-order term rewriting system, with scope indices erased, and hand the resulting 75 rules to external provers. AProVE proves termination in 23.9 s and confluence in 24.9 s. Local confluence plus termination gives confluence by Newman's lemma, so the two results agree by independent routes, and the obligations of section 3.1 are discharged.

The system does *not* pass Agda's global `--confluence-check`, which reports 277 errors, every one of them **RewriteAmbiguousRules** and none a non-joinable pair. That check asks that no two left-hand sides overlap at all, which no σ -calculus can satisfy: **assoc** overlaps every rule whose left-hand side mentions $\circ$, by construction. Those overlaps are exactly the critical pairs the local check examines, and it joins every one of them.

Both checks certify the rule set as it is seen from outside the **opaque** blocks, where the operations do not reduce. An **unfolding** block puts the defining clauses back into the reduction relation, and neither check covers that combination; the proofs of the rules live in such a block, but they are discharged before any rule is registered. The certificate therefore carries an obligation the type checker cannot enforce: once a rule is in scope, its operations must never be unfolded again (section 3.4). **abstract** does not serve here, although it looks like the stronger seal. Agda's confluence checker reduces through it deliberately, so that rewriting cannot break subject reduction inside an abstract block, and the same development declared abstract reports 197 non-joinable critical pairs.

Cost. Definitional equalities are paid for at type-checking time. Measured on one core under Agda 2.8, a module that does nothing but import a generated core costs about 70 s, against 1 s for a comparable module importing only the standard library. The figure is flat in the size of the rule set, 69 s at 77 rules and 66 s at 111. Per-module times across the development run from 74 to 120 s, so that fixed import cost

dominates and the proof content is cheap. The Part 3 animation is the exception, at about 76 minutes, and there the rules are doing evaluation instead of conversion.

Two controls make the verdicts meaningful. Both properties hold at two signature sizes, three and eight constructors, which is what licenses the generator claim. And the setup has a falsifier that fires, because adding the two η -laws is refuted for confluence by three independent tools, as section 3.2 predicts. Evidence at two sizes is not a proof schematic in the signature, and obtaining one would mean decomposing the system along the lines of compositional confluence criteria [35].

5 Implementation and Evaluation

A construction shown on one language is a claim about that language. This section turns it into a claim about the method. We generate the infrastructure from a syntax description (section 5.1) and measure what it buys on the POPLmark Challenge (section 5.2).

5.1 Code Generation

Nothing in section 3 is specific to System F. Of the seven groups in table 1, only traversal depends on the signature, and it depends on it in the simplest possible way, two rules per constructor. Everything else is schematic in the constructor list, so the whole construction can be generated. We provide a script that turns a higher-order abstract syntax (HOAS) description into Agda code with the rules ready for rewriting. Figure 5 is the description of our running example, and it is the entire input: eleven constructors across three sorts, with binders written as function arguments. AUTOSUBST 2 does the same for Rocq, emitting the scoped syntax together with the substitution lemmas that need induction over it.

Several of AUTOSUBST 2's features come for free. Vector substitutions are unnecessary and mutually recursive definitions work unchanged, both because the syntax is multi-sorted. Inline Agda code, such as lists or products of terms, can be encoded with additional sorts at the cost of standard library support. Two features are missing. A distinction between sorts with and without binders would take an additional index on `Sort`. Variadic syntax could be encoded through arguments to the sort constructors [31].

Adding a constructor to a language is mechanical, because only the traversal group depends on it and it contributes exactly two rules there. Binders that bind several variables at once are not free in the same way. They are handled by a lifting operation that lifts by a *list* of sorts, and the completion has to be redone at each lifting level, which is what the iterated $\uparrow^*$ family costs. This is why the generated cores carry more rules than the 72 of section 3.3: 77 for STLC, 83 with sums, 87 for $F_{<}$, 101 with records and 111

with pattern matching. Everything above 72 is the $\uparrow^*$ family and the extra traversal rules of the larger signatures.

The script is a single Python file with strict type checking enabled, comprising a parser for HOAS descriptions and ad-hoc code generation, and it carries all boilerplate Agda code so that others can run it standalone. Beside the five languages of section 5.2 we generate a dozen files from the HOAS descriptions shipped with AUTOSUBST 2, which is the cheapest evidence we have that the schema is not tuned to our own examples.

5.2 POPLmark and POPLmark Reloaded

To test the construction at a scale where substitution reasoning hurts, we solved the POPLmark Challenge, Parts 1A, 1B, 2A, 2B and 3, and POPLmark Reloaded including its sums extension, on top of generated cores. The development is roughly 9,100 lines of Agda, 6,400 of them excluding blanks and comments, of which 2,800 are generated infrastructure and 3,500 hand-written metatheory. It spans five languages, $F_{<}$, $F_{<}$ with records, $F_{<}$ with pattern matching, STLC and STLC with sums, and `fun-ext` is its only postulate.

The point of comparison is the substitution reasoning a de Bruijn development normally pays for. In the Rocq solutions by the AUTOSUBST authors the σ -laws are applied by an `asimpl` tactic, invoked explicitly wherever substitutions must be normalized. Here there is no such step, because normalization happens during conversion. Section 6 places the coverage among published solutions. The claim rests on three measurements, each a property of the whole development rather than of a chosen example.

- The η -laws, the ones we cannot register, are applied by hand zero times. Their absence is not felt.
- No proof uses a `rewrite` clause. Thirty transports remain across the metatheory modules, of which exactly two are along a substitution equation, and both are the same lemma.
- That lemma, which we call `[]-as-ren`, is a single law, and its failure is not an accident of the tooling. It says

$$t [\text{' } x]_0 = t [x \cdot \text{id}]^R$$

and the two sides are two *distinct* normal forms. The left one unfolds to $t [(\text{' } x) \cdot \text{id}]$, which is cons-shaped, while coincidence needs a syntactic $\langle \xi \rangle$ to collapse into the renaming world. The rule that would bridge them is $(\text{' } x) \cdot \langle \xi \rangle \rightarrow \langle x \cdot \xi \rangle$, in the same $S \rightarrow R$ direction as the rest of the coercion group. Registering it costs four non-joinable pairs, and the one that survives every attempt to complete the system is

$$(x [\uparrow \xi]) [(\text{' } y) \cdot \langle \xi_1 \rangle]$$

```

1101 kind : Type
1102 type : Type
1103 expr : Type
1104
1105 -- the constructors for kind
1106 star : kind
1107
1108 -- the constructors for type
1109 arr : type -> type -> type
1110 all : kind -> (type -> type) -> type
1111
1112 -- the constructors for expr
1113 app : expr -> expr -> expr
1114 lam : type -> (expr -> expr) -> expr
1115 tapp : expr -> type -> expr
1116 tlam : kind -> (type -> expr) -> expr

```

Figure 5. The higher-order abstract syntax description of System F that the generator consumes. Sorts are declared as Type, and a binder appears as a function argument, so `lam` binds one `expr` in its body. From this the generator emits the syntax of fig. 1 and the rule set of section 3.3

whose reducts meet only if lift-cons^R can fire, which needs composition at a variable to fold. It pushes, because folding there overlaps $x \text{ [wk]} = \text{suc } x$ unjoinably. The one hand-proved substitution fact is therefore the push-at-V decision seen from the substitution side, not an independent gap, and we supply the missing join as an induction on the term.

The system leaves two costs in place. First, a function defined by recursion over the syntax does not commute with substitution for free. The size and shape measures the Challenge needs still take their own induction, because with an abstract argument the traversal rules do not fire. Second, a rewrite system constrains what may be matched as well as what may be proved. An earlier version of our transitivity proof for subtyping recursed on $Q \text{ [}\xi\text{]}^R$ to make the induction structural in Q . It type checks in the ordinary sense and is then rejected by `--local-confluence-check`. Matching on Q puts $(\forall \dots) \text{ [}\xi\text{]}^R$ into the clause's left-hand side, where it overlaps a traversal rule unjoinably.

6 Related Work

Three lines of work meet here. Solutions to the POPLmark Challenge fix the benchmark our evaluation is measured against, generic and generated syntax libraries are what we would replace, and a small recent literature attacks the same tension between provability and definitional behaviour from the semantic side.

POPLmark solutions. Fifteen solutions to the Challenge have been published. Of those, four ever attempted the records extension, three the pattern-matching extension, and four Part 3. Berghofer's Isabelle/HOL de Bruijn development is the only one covering Parts 1, 2 and 3 together, and it animates the congruence-rule presentation because its code generator cannot execute the challenge's evaluation contexts. We cover 1A, 1B, 2A and 2B, and answer Part 3 as a decision procedure for the evaluation-context relation, machine-checking all seven tests of the challenge's own

step.poplmark, at the cost of running on 2B without the pattern `let`. On POPLmark Reloaded we join Allais's generic-syntax Agda solution, Beluga and the two Rocq AUTOSUBST solutions as a complete STLC and STLC+ solution. Allais additionally proves completeness of the inductive characterization of strong normalization, which we do not. The nearest neighbours in technique are Schäfer and Stark's Rocq de Bruijn solutions, by the authors of AUTOSUBST and of the Reloaded benchmark. Coverage is what makes the claim about the substitution reasoning underneath it testable.

We have discussed AUTOSUBST and related research [34, 37, 38], user-defined rewrite rules [14, 15, 25] and opaque definitions [19] throughout this work.

In recent work on local rewrite rules [26], the authors also rewrite σ -calculus rules with support for first-class renamings in Rocq. They too model renamings and substitutions as functions, but cannot rewrite all laws against that model, because with local rewrite rules the instantiation of the equations must hold definitionally rather than propositionally to maintain consistency. Rocq also has global rewrite rules [25], and we expect our approach to translate Rocq, however, cannot instantiate rewrite rules with proofs, so the consistency guarantee would be lost.

Definitional substitution without rewriting. Kaposi and Pujet [23] meet the same tension from the semantic side. They present the initial category with families so that substitutions are metatheoretical functions, which makes the equations of the substitution calculus hold by computation and removes the transports that an indexed-inductive presentation forces. The mechanism is the one we exploit, namely that a function model computes, but it is deployed without rewrite rules, so which equations become definitional is fixed by the encoding rather than chosen. The η -laws are not among them, which is the boundary we reach from the syntactic side. Chen, Nordvall Forsberg and Tsai [13] give an intrinsic representation of type theory in Cubical Agda as quotient inductive-inductive-recursive types, and obtain a syntax that

needs no transports, no postulates and no custom rewrite rules. Where they replace the rewrite rules by a quotient, we keep the plain inductive type and pay for it with a confluence obligation.

Generators for binder boilerplate predate and accompany AUTOSUBST. Needle & Knot [24] generates the boilerplate for complex binding forms, including the relational lemmas that AUTOSUBST leaves out, but does not aim at an equational theory of substitution and so has no equations to turn into rules. Strongly typed term representations in Rocq [10] are the ancestor of the scoped syntax we use.

Another line of research concerns generic programming approaches [4, 5] that build syntax from general building blocks supporting sums, products and binders, and thereby abstract over renamings and substitutions. The approach carries over to a scoped, multi-sorted setting [31]. We believe generic programming and rewriting the σ -calculus are orthogonal. Applying our rule set to the universe of syntaxes with binding would remove the need for external code generation altogether.

The ABSTRACT BINDING TREES library for Agda shares a goal with AUTOSUBST and is used, for instance, in Siek's work [36]. It builds an object language from a generic syntax, but supports only extrinsic scoping and single-sorted syntax, and focuses on generic theorems about substitution. Its experimental support for rewriting with σ -calculus laws misses rules and does not pass the confluence check.

Wadler experiments with explicit substitutions and implements weakening as a constructor, thereby avoiding renamings [41]. He rewrites some of the σ -calculus laws but does not pass the confluence checker. Section 1 discussed what that representation costs.

Wadler and colleagues also investigate a uniform treatment of renamings and substitutions in a simply typed setting [6], originating in work by McBride [27]. It unifies repeated similar lemmas, merging all four compositionality lemmas into one, by hiding the structural dependence of renamings and substitutions in a uniform, *Mode*-indexed vector, and lets the termination checker see that dependence where it matters. Saffrich [31] instead hides it behind instance resolution, in a scoped setting where the uniform structure is again a function, and adds reflection-based code generation for the syntax-dependent lemmas, which could be adopted here in place of our external generator.

Adapting our approach to a setting where renamings and substitutions are treated uniformly is a logical next step, but the equational theory would have to be extended in non-obvious ways to support uniform proofs over both. It would also require a two-level rewrite system, one on the type level, normalizing expressions in the lattice of *Modes* generated by $\mathbb{V} \sqsubseteq \mathbb{T}$ [6], and one for the adapted σ -calculus. In our experiments the interaction of the two layers made confluence hard to preserve.

7 Future Work

Opaque Definitions for Rewriting

The construction suggests a general recipe for integrating normalization for suitable algebraic structures into a proof assistant: define operations for the structure; establish a confluent rewrite system over them; instantiate the structure with an underlying model and prove the laws inside *opaque* blocks; and, with the model hidden, *REWRITE* the equations.

Classical sets are one candidate. Reasoning about them in Agda usually means lists up to permutation together with a uniqueness predicate, which is tedious and generates a great deal of uninteresting code. Their characteristic function supplies the model the recipe asks for. Sets become $\mathcal{S} = \mathcal{A} \rightarrow \text{Bool}$ for some $\mathcal{A} : \text{Set } \ell$, the operations become opaque functions

```
opaque
  0 : S
  0 a = false
  _⊔_ : S → S → S
  (A ⊔ B) a = A a ∨ B a
```

and laws such as $(A \cup \emptyset) \equiv A$ become provable and registrable. The boolean ring interpretation [22] is a natural starting point for the rule set, but it needs rewriting modulo associativity and commutativity. Dedukti [8] supports this and Agda does not, which is the missing ingredient.

Implications for Intrinsically Typed Syntax

The construction applies as it stands to intrinsically typed syntax whose type index lives in *Set*. Richer syntax, where a substitution appears in the type index, is out of its reach for now, and that is where the phenomenon known as transport hell is worst. The transfer lemma (*subst* in Agda) then has to carry every step of every proof. Intrinsically typed System F has been formalized in Agda before, executably and in full [11], and a binary logical relation over it [32] is a close example of the cost. Type-in-type and expression-in-expression reasoning stays easy, but type-in-expression substitution also affects the index of the expression substituted into, and the authors spend thousands of lines on the resulting transports. Rewriting the σ -calculus rules on the syntax of the typing index should remove them, and should generalize to other intrinsically typed formalizations.

8 Conclusion

We have turned the normalization procedure of the σ -calculus into definitional equalities in Agda. The operations must compute, so that the laws are provable, and stay inert, so that the rules match. Opaque definitions give both.

The theory we register is $\sigma_{\#}$ instantiated once per world, with the rules that first-class renamings force between the two. Which laws survive is forced, because native inductive variables and surjective pairing cannot coexist in one rewrite

system. Agda certifies local confluence and AProVE termination, a generator emits it from a syntax description, and the POPLmark challenges exercise it at scale.

References

- [1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. 1989. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL '90). Association for Computing Machinery, New York, NY, USA, 31–46. doi:10.1145/96709.96712
- [2] Andreas Abel and Brigitte Pientka. 2011. Higher-Order Dynamic Pattern Unification for Dependent Types and Records. In *Typed Lambda Calculi and Applications*, Luke Ong (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 10–26.
- [3] Robin Adams. 2004. Formalized metatheory with terms represented by an indexed family of types. In *Proceedings of the 2004 International Conference on Types for Proofs and Programs* (Jouy-en-Josas, France) (TYPES'04). Springer-Verlag, Berlin, Heidelberg, 1–16. doi:10.1007/11617990_1
- [4] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. 2021. A Type and Scope Safe Universe of Syntaxes with Binding: Their Semantics and Proofs. arXiv:2001.11001 [cs.PL] <https://arxiv.org/abs/2001.11001>
- [5] Guillaume Allais, James Chapman, Conor McBride, and James McKinna. 2017. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs* (Paris, France) (CPP 2017). Association for Computing Machinery, New York, NY, USA, 195–207. doi:10.1145/3018610.3018613
- [6] Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. 2025. Substitution without copy and paste. <https://nottingham-repository.worktribe.com/output/51884690/substitution-without-copy-and-paste>. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)* (2025-07-19). ACM, Birmingham, UK, 17 pages.
- [7] Thorsten Altenkirch and Ambrus Kaposi. 2016. Type theory in type theory using quotient inductive types. *SIGPLAN Not.* 51, 1 (Jan. 2016), 18–29. doi:10.1145/2914770.2837638
- [8] Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. 2023. Dedukti: a Logical Framework based on the $\lambda\Pi$ -Calculus Modulo Theory. arXiv:2311.07185 [cs.LO] <https://arxiv.org/abs/2311.07185>
- [9] Franz Baader and Tobias Nipkow. 1998. *Term Rewriting and All That*. Cambridge University Press, Cambridge. doi:10.1017/CBO9781139172752
- [10] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *Journal of Automated Reasoning - JAR* 49 (08 2012), 1–19. doi:10.1007/s10817-011-9219-0
- [11] James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. 2019. System F in Agda, for Fun and Profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 255–297. doi:10.1007/978-3-030-33636-3_10
- [12] Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2
- [13] Liang-Ting Chen, Fredrik Nordvall Forsberg, and Tzu-Chun Tsai. 2026. Can We Formalise Type Theory Intrinsically without Any Compromise? A Case Study in Cubical Agda. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Rennes, France) (CPP '26). Association for Computing Machinery, New York, NY, USA, 201–215. doi:10.1145/3779031.3779090
- [14] Jesper Cockx. 2020. Type Theory Unchained: Extending Agda with User-Defined Rewrite Rules. In *25th International Conference on Types for Proofs and Programs (TYPES 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 175)*, Marc Bezem and Assia Mahboubi (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1–2:27. doi:10.4230/LIPIcs.TYPES.2019.2
- [15] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341
- [16] Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. 1996. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM* 43, 2 (March 1996), 362–397. doi:10.1145/226643.226675
- [17] N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0
- [18] Jean-Yves Girard. 1972. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. PhD thesis. Éditeur inconnu. <https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf>
- [19] Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. 2025. Controlling unfolding in type theory. *Mathematical Structures in Computer Science* 35 (2025), e38, 1–37. doi:10.1017/S0960129525100327
- [20] Thérèse Hardin, Luc Maranget, and Bruno Pagano. 1998. Functional runtime systems within the lambda-sigma calculus. *Journal of Functional Programming* 8, 2 (1998), 131–176. doi:10.1017/S0956796898003001
- [21] Martin Hofmann. 1997. Syntax and semantics of dependent types. In *Extensional Constructs in Intensional Type Theory*. Springer, London, 13–54.
- [22] Jieh Hsiang. 1985. Refutational theorem proving using term-rewriting systems. *Artificial Intelligence* 25, 3 (1985), 255–300. doi:10.1016/0004-3702(85)90074-8
- [23] Ambrus Kaposi and Loïc Pujet. 2025. Type Theory in Type Theory using a Strictified Syntax. *Proc. ACM Program. Lang.* 9, ICFP, Article 266 (Aug. 2025), 31 pages. doi:10.1145/3747535
- [24] Steven Keuchel, Stephanie Weirich, and Tom Schrijvers. 2016. Needle & Knot: Binder Boilerplate Tied Up. In *Proceedings of the 25th European Symposium on Programming Languages and Systems - Volume 9632*. Springer-Verlag, Berlin, Heidelberg, 419–445. doi:10.1007/978-3-662-49498-1_17
- [25] Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. 2024. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:18. doi:10.4230/LIPIcs.ITP.2024.26
- [26] Yann Leray and Théo Winterhalter. 2026. Encode the Cake and Eat It Too: Controlling Computation in Type Theory, Locally. *Proc. ACM Program. Lang.* 10, POPL, Article 62 (Jan. 2026), 30 pages. doi:10.1145/3776704
- [27] Conor McBride. 2005. Type-preserving renaming and substitution. (2005). <http://strictlypositive.org/ren-sub.pdf> Unpublished manuscript.
- [28] M. H. A. Newman. 1942. On theories with a combinatorial definition of “equivalence”. *Annals of Mathematics* 43, 2 (1942), 223–243. doi:10.2307/1968867
- [29] Frank Pfenning and Conal Elliott. 1988. Higher-order abstract syntax. In *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI '88)*. Association for Computing Machinery, New York, NY, USA, 199–208. doi:10.1145/53990.54010

- [30] John C. Reynolds. 1974. Towards a theory of type structure. In *Programming Symposium*, B. Robinet (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 408–425. <https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf>
- [31] Hannes Saffrich. 2024. Abstractions for Multi-Sorted Substitutions. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 32:1--32:19. doi:10.4230/LIPIcs.ITP.2024.32
- [32] Hannes Saffrich, Peter Thiemann, and Marius Weidner. 2024. Intrinsically Typed Syntax, a Logical Relation, and the Scourge of the Transfer Lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development (Milan, Italy) (TyDe 2024)*. Association for Computing Machinery, New York, NY, USA, 2–15. doi:10.1145/3678000.3678201
- [33] Steven Schäfer, Gert Smolka, and Tobias Tebbi. 2015. Completeness and Decidability of de Bruijn Substitution Algebra in Coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs (Mumbai, India) (CPP '15)*. Association for Computing Machinery, New York, NY, USA, 67–73. doi:10.1145/2676724.2693163
- [34] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn Terms and Parallel Substitutions. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 359--374. https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf
- [35] Kiraku Shintani and Nao Hirokawa. 2024. Compositional Confluence Criteria. *Logical Methods in Computer Science* Volume 20, Issue 1 (2024), 6:1--6:33. doi:10.46298/lmcs-20(1:6)2024
- [36] Jeremy G. Siek and Tianyu Chen. 2021. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming* 31 (2021), e30. doi:10.1017/S0956796821000241
- [37] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq*. Ph.D. Dissertation. Saarland University, Saarbrücken, Germany. https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.).
- [38] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs (Cascais, Portugal) (CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3293880.3294101
- [39] Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. 2013. Nominal 2. *Archive of Formal Proofs* 2013 (February 2013), 1 pages. <https://isa-afp.org/entries/Nominal2.html>, Formal proof development.
- [40] Christian Urban and Christine Tasson. 2005. Nominal techniques in Isabelle/HOL. In *Proceedings of the 20th International Conference on Automated Deduction (Tallinn, Estonia) (CADE' 20)*. Springer-Verlag, Berlin, Heidelberg, 38–53. doi:10.1007/11532231_4
- [41] Philip Wadler. 2024. Explicit Weakening. *Electronic Proceedings in Theoretical Computer Science* 413 (Nov. 2024), 15–26. doi:10.4204/eptcs.413.2